

USACH

SISTEMA DE ADMINISTRACIÓN ACADÉMICA UNIVERSITARIA

Start

PROFESOR:
MATÍAS CALDERÓN



AYUDANTE:
FERNANDO SOLÍS



Integrantes



Inicio Desarrollo Final

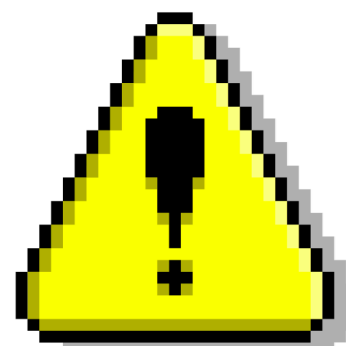
 B1 : VICENTE
TAPIA

 B2 : DANIEL
DÍAZ

 F1 : BENJAMÍN
ARAYA

 F2 : JORDAN
GONZÁLEZ





Para esta tercera etapa, la universidad además necesita incorporar un **motor NoSQL** orientado a documentos: **MongoDB**, desplegado como Replica Set, para garantizar transacciones ACID multi-documento en procesos críticos como la inscripción. El sistema debe validar la integridad de los datos mediante **Schema Validation**, calcular **tasas de aprobación** y reprobación **mediante Aggregation Pipelines**, y mantener un certificado de notas actualizado reactivamente mediante **Change Streams** cada vez que se registra una calificación.



¿Qué debemos desarrollar?



Inicio Desarrollo Final

- **Modelado**: las notas se almacenan en grades y referencian una inscripción; no se guardan dentro del estudiante.
- **Schema Validation**: valida campos obligatorios, tipos de datos, rangos y estructura de los documentos.
- **Transacción**: comprueba estudiante, prerrequisitos, semestre y cupo; luego descuenta el cupo y crea la inscripción.
- **Analítica**: calcula aprobación y reprobación mediante \$group y \$bucket.
- **Índices**: evita inscripciones duplicadas y permite buscar asignaturas por texto.
- **Reactividad**: actualiza certificados mediante \$merge y Change Streams.



Descripción avance de proyecto



Inicio Desarrollo Final

- La persistencia del sistema fue rediseñada completamente utilizando **MongoDB** y un **Replica Set**.
- Se incorporaron validaciones de **documentos** y **transacciones** seguras para controlar cupos, prerequisites e inscripciones.
- Se agregaron **índices**, búsqueda de texto y reportes de aprobación y reprobación mediante **Aggregation Pipelines**.
- Se implementaron certificados de notas materializados, actualizados automáticamente mediante **Change Streams**.
- Además, se mejoraron las **interfaces** por rol y se corrigieron inconsistencias en horarios, cursos, inscripciones y asignaturas.



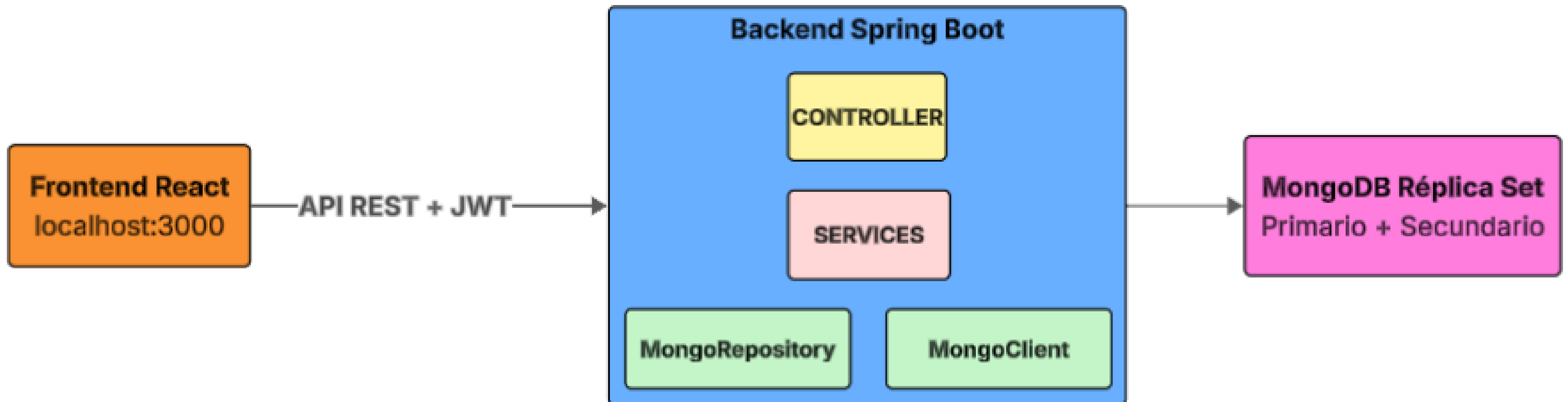
Arquitectura



Inicio **Desarrollo** Final



MONOLÍTICA con
MongoDB



Inicio **Desarrollo** Final



DATABASE

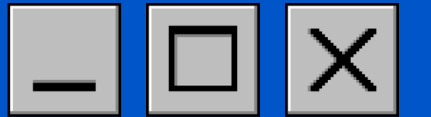
- MongoDB 7.0
- Replica Set: primario y secundario
- Schema Validation
- Aggregation Pipelines
- Change Streams



mongoDB®



Tecnologías Usadas



Inicio **Desarrollo** Final



BACKEND



v4.0.6



Dependencias usadas:

- spring-boot-starter-web
- spring-boot-starter-data-jdbc
- spring-boot-starter-security
- postgresql
- lombok
- springdoc-openapi (Swagger UI)
- spring-boot-starter-test
- jjwt-api / jjwt-impl / jjwt-jackson



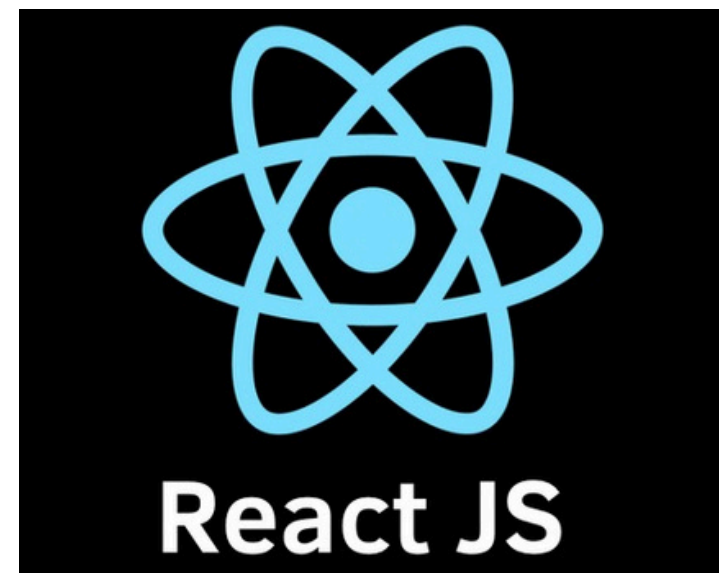
21 jdk



json web
token



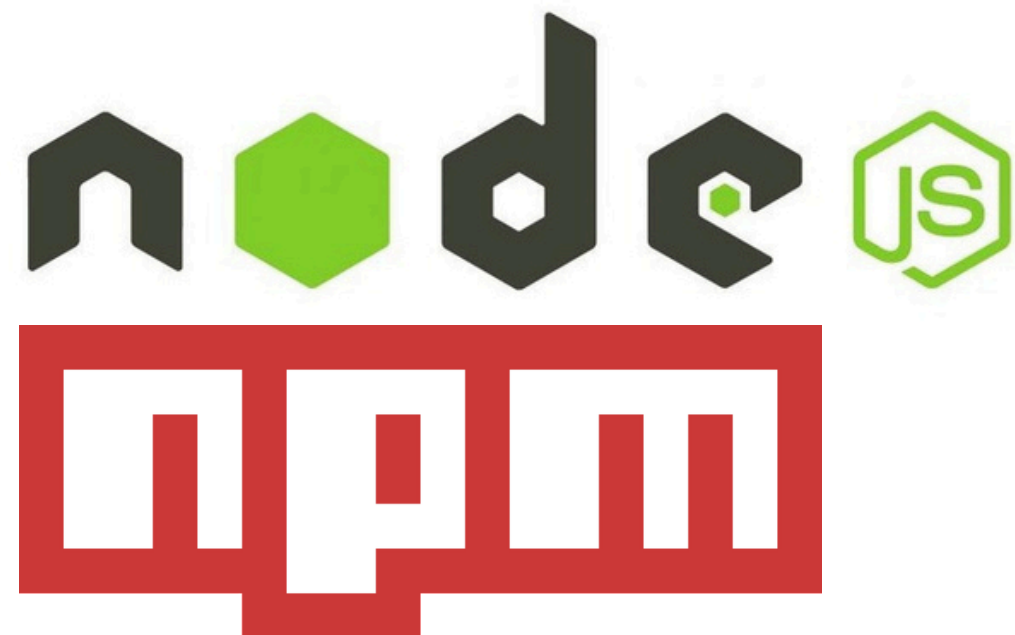
Inicio Desarrollo Final



◀◀◀
v18.x



FRONTEND



◀◀◀
v22.22.0

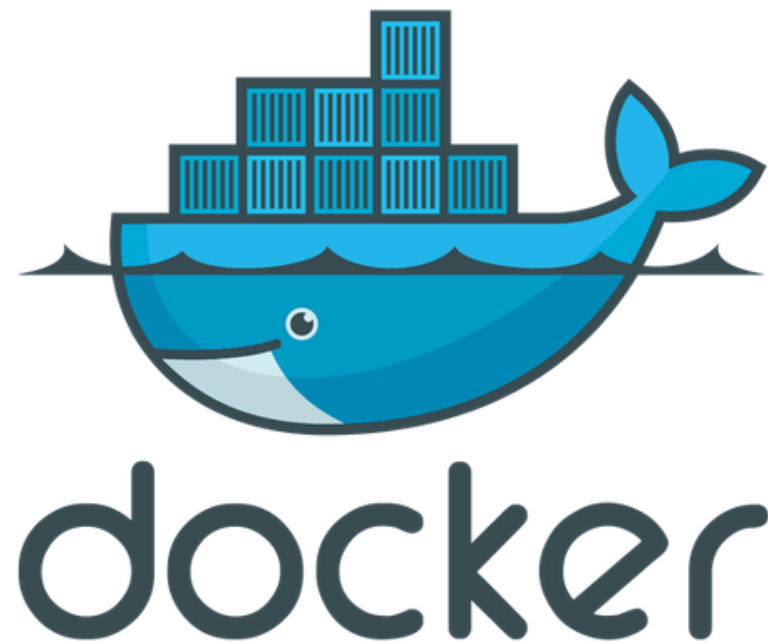
◀◀◀
v10.9.4

Librerías utilizadas en frontend:

- Bootstrap (estilo y diseño)
- Axios (HTTP → backend)
- React Router DOM (componentes)



Inicio **Desarrollo** Final



- Maven
- Node.js 22



DESPLIEGUE



Contribuciones



Inicio

Desarrollo

Final

Active branches

Branch	Updated	Check status	Behind	Ahead	Pull request
lab3	2 hours ago		0	0	
feature/f1-enrollment-ui	4 hours ago		2	1	
feature/f2-reports-ui	10 hours ago		5	0	
feature/b2-mongo-analytics	5 days ago		11	0	#12
feature/b1-mongo-core	last week		13	0	#11

Branches

Tags

lab2-final

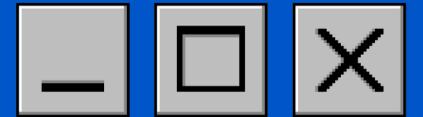
lab1-final

[View all tags](#)



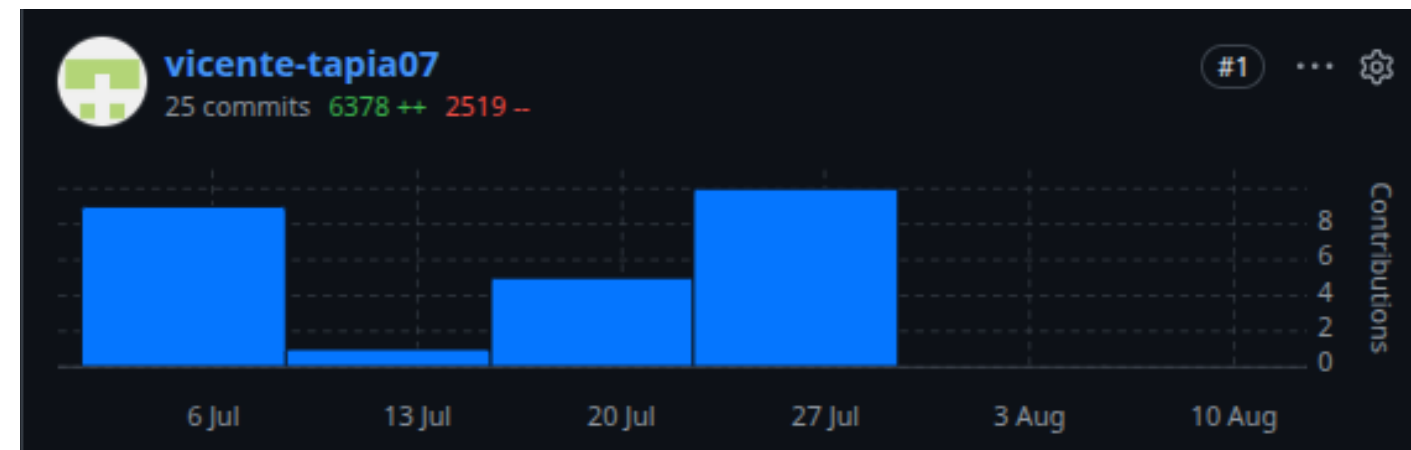
**TRABAJO REALIZADO
POR RAMAS.**





Inicio **Desarrollo** Final

Contribuciones del ultimo mes



B1 – Mongo Core + Inscripción
transaccional

Colecciones principales:

students, sections, enrollments, grades

Archivos principales:

- MongoConfig.java
- EnrollmentDocument.java
- SectionDocument.java
- MongoEnrollmentRepository.java
- EnrollmentTransactionService.java
- EnrollmentController.java
- 1_collections_and_schema.js

Funcionalidades implementadas:

- Conexión al Replica Set.
- Modelado mediante embedding y referencias.
- Validación estricta con \$jsonSchema.
- Inscripción multi-documento ACID.
- Validación de estudiantes, cupos y prerequisites.
- Cancelación, devolución del cupo y reinscripción.

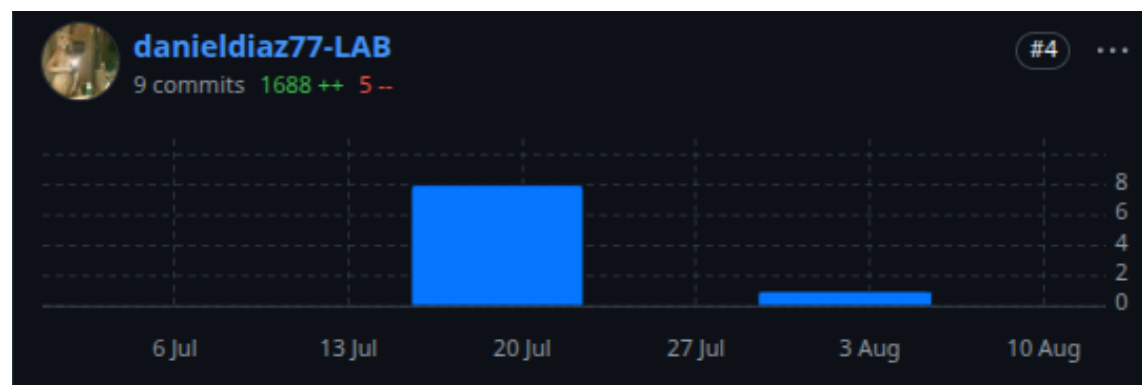
Endpoints clave:

- POST /api/enrollments/enroll
- DELETE /api/enrollments/{id}
- GET
/api/enrollments/student/{studentId}



Inicio **Desarrollo** Final

Contribuciones del ultimo mes



B2 – Mongo Analytics + Reactividad

Colecciones principales:

grades, certificados_notas, audit_logs

Archivos principales:

- ReportAggregationService.java
- CertificateChangeStreamService.java
- MongoReportController.java
- 2_indexes.js
- 3_seed_mongo.js
- 4_certificates_merge.js

Funcionalidades implementadas:

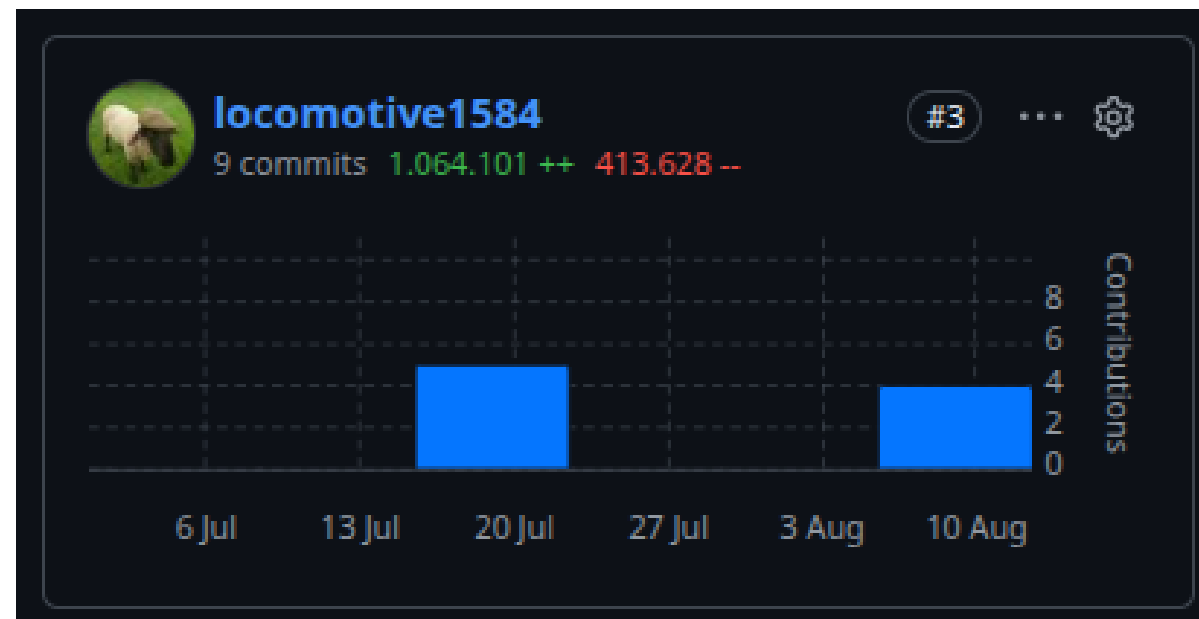
- Reportes mediante \$group y \$bucket.
- Índices únicos, compuestos, de texto y TTL.
- Buscador de asignaturas.
- Certificados materializados mediante \$merge.
- Actualización automática con Change Streams.
- Creación del mock académico.

Endpoints clave:

- GET /api/mongo/reports/pass-fail-rate
- GET /api/mongo/subjects/search
- GET /api/mongo/certificates/{studentId}



Contribuciones del ultimo mes



B2 – Interfaz de inscripciones

Vistas principales:

EnrollForm.jsx, StudentEnrollments.jsx

Funcionalidades implementadas:

- Visualización de secciones disponibles.
- Inscripción de asignaturas.
- Cancelación de inscripciones.
- Historial separado por estado.
- Inscripciones activas, completadas y canceladas.
- Mensajes de éxito y errores académicos

Endpoints clave:

- GET /api/sections
- GET /api/enrollments/student/{studentId}
- POST /api/enrollments/enroll
- DELETE /api/enrollments/{id}



Inicio **Desarrollo** Final

Contribuciones del ultimo mes



B2 – Interfaz de inscripciones

Vistas principales:

MongoTabs.jsx, PassFailReport.jsx, StudentCertificate.jsx, SubjectSearch.jsx

Funcionalidades implementadas:

- Reporte de aprobación y reprobación.
- Filtros por asignatura y semestre.
- Distribución de notas por rangos.
- Visualización del certificado académico.
- Buscador de asignaturas por texto.
- Acceso diferenciado según el rol.

Endpoints clave:

- GET /api/mongo/reports/pass-fail-rate
- GET /api/mongo/certificates/{studentId}
- GET /api/mongo/subjects/search

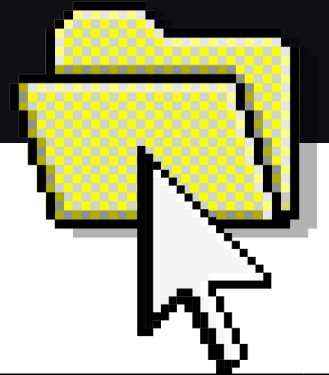




TRABAJO DE B1

¿Cómo generaría una colección materializada con el certificado de notas de cada estudiante, actualizada reactivamente mediante Change Streams cada vez que se registra una nueva calificación?

```
Document reservedSection = sections.findOneAndUpdate(
    session,
    and(
        eq("_id", sectionId),
        eq("status", "OPEN"),
        gt("availableSeats", 0)
    ),
    inc("availableSeats", -1),
    new FindOneAndUpdateOptions().returnDocument(ReturnDocument.AFTER)
);
if (reservedSection == null) {
    throw new EnrollmentTransactionException(
        NO_AVAILABLE_SEATS,
        "El último cupo fue tomado por otra inscripción"
    );
}
```





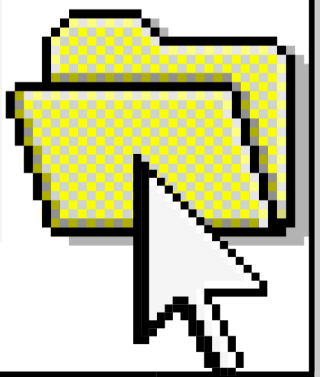
TRABAJO DE B2

¿Cómo generaría una colección materializada con el certificado de notas de cada estudiante, actualizada reactivamente mediante Change Streams cada vez que se registra una nueva calificación?

```
while (running) {
  try (var cursor = grades.watch/watchPipeline()
      .fullDocument(FullDocument.UPDATE_LOOKUP)
      .cursor()) {

    while (cursor.hasNext()) {
      ChangeStreamDocument<Document> event = cursor.next();
      ObjectId studentId = event.getFullDocument()
          .getObjectId("studentId");
      rebuildCertificate(studentId);
    }
  }

  // rebuildCertificate: $match -> $lookup -> $group -> $merge
  pipeline.add(Aggregates.merge("certificados_notas",
      new MergeOptions()
          .whenMatched(MergeOptions.WhenMatched.REPLACE)
          .whenNotMatched(MergeOptions.WhenNotMatched.INSERT)));
  grades.aggregate(pipeline).toCollection();
}
```



Código implementado

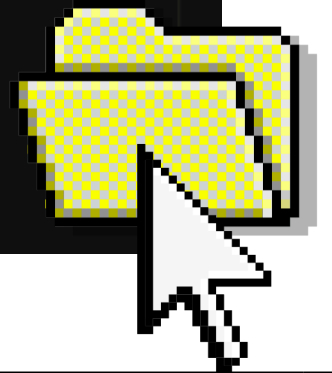


Inicio **Desarrollo** Final

i

TRABAJO DE F1

```
const handleEnroll = async (sectionId) => {
  if (!studentId || !sectionId) return;
  setEnrollingId(sectionId);
  setError(''); setSuccess('');
  try {
    await api.post('/api/enrollments/enroll', {
      studentId,
      sectionId,
    });
    setSuccess('¡Inscripción exitosa!');
    setTimeout(() => navigate('/my-enrollments'), 1500);
  } catch (err) {
    const msg = (err.response?.data?.message ?? err.response?.data ?? '').toString().toLowerCase();
    if (msg.includes('prerequisit') || msg.includes('requisito')) {
      setError('No cumples los prerrequisitos para esta asignatura.');
    } else if (msg.includes('cupo') || msg.includes('slot') || msg.includes('seat')) {
      setError('No hay cupos disponibles en esta sección.');
    } else if (err.response?.status === 409) {
      setError('Ya estás inscrito en esta asignatura.');
    } else {
      setError(err.response?.data?.toString() || 'No se pudo completar la inscripción.');
    }
  } finally {
    setEnrollingId(null);
  }
};
```



Inicio Desarrollo Final

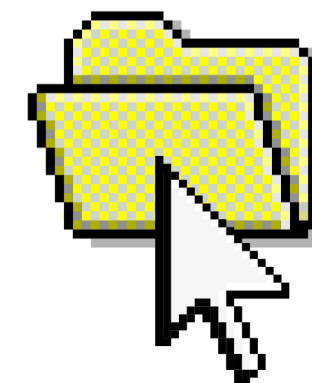


TRABAJO DE F2

```
const fetchReport = useCallback(async (subject, semester) => {
  setLoading(true);
  setError("");
  try {
    const params = {};
    if (subject) params.subjectId = subject;
    if (semester) params.semesterId = semester;

    const res = await api.get("/api/mongo/reports/pass-fail-rate", {
      params,
    });
    const items = res.data?.bySubjectAndSemester ?? [];
    const buckets = res.data?.gradeDistribution ?? [];

    setRates(items);
    setDistribution(buckets);
    setLastRefresh(new Date().toLocaleTimeString("es-CL"));
    return items;
  } catch (err) {
    setError(
      readApiError(
        err,
        "No se pudo ejecutar el pipeline de agregación en MongoDB.",
      ),
    );
    setRates([]);
    setDistribution([]);
    return [];
  } finally {
    setLoading(false);
  }
}, []);
```

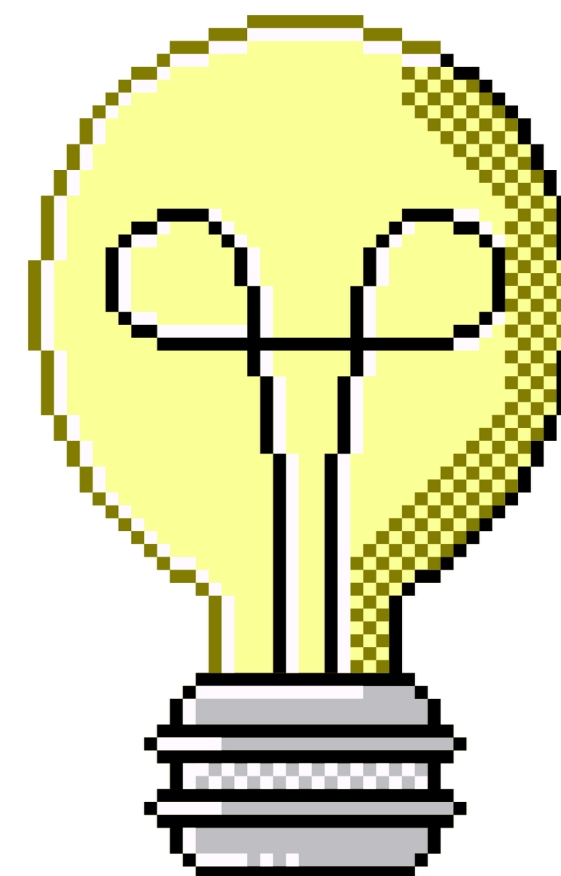


Demostración



Inicio **Desarrollo** Final

**HORA DE BOOTEAR EL
PROYECTO**



MUCHAS GRACIAS



Inicio Desarrollo **Final**

CONCLUSIÓN

Sistema académico integrado completamente con MongoDB.

Inscripciones y cupos protegidos mediante transacciones.

Reportes y certificados generados eficientemente.

Aprendizaje: NoSQL también necesita validación, estructura y consistencia.

